

Navegación

Taiwán posee una gran cantidad de montañas muy altas, con más de 250 de ellas superando los 3,000 metros de altura. A-Ming planea construir redes robóticas de entrega por medio de tirolesas dentro de la Cadena de Montañas de Taiwán para transportar paquetes entre aldeas remotas.

En el plan de A-Ming, cada red consiste de N nodos, numerados del 0 a $N - 1$. Dentro de estos nodos, **exactamente** $K = 6$ de ellos son estaciones de recarga, y todos los $N - K$ nodos restantes son aldeas. Los nodos están conectados mediante $N - 1$ cables numerados del 0 a $N - 2$ que pueden recorrerse **en ambas direcciones**. Para cada i ($0 \leq i \leq N - 2$), el cable i conecta a los nodos $U[i]$ y $V[i]$. Cada cable conecta dos nodos distintos, y cada par de nodos está conectado por a lo más un cable. Se garantiza que es posible moverse entre cualquier par de nodos utilizando los cables.

La **distancia** entre dos nodos a y b denotada por $d(a, b)$ se define de la siguiente manera:

- Si $a = b$ entonces $d(a, b) = 0$.
- En caso contrario, $d(a, b)$ es igual a la mínima cantidad de cables necesarios para llegar desde a hasta b .

El **grado** de un nodo se define como la cantidad de cables conectados a ese nodo en específico. Además se dice que un nodo es **hoja** si su grado es exactamente 1. A-Ming conoce un entero positivo B tal que todos los nodos **no hojas** tienen grado al menos B .

A-Ming ha preparado 100 tipos distintos de cables, numerados $0, 1, \dots, 99$. Cada cable en la red será de alguno de esos tipos.

Los robots se moverán a través de las tirolesas para cumplir con las entregas necesarias. A-Ming desea instalar un plan de navegación que ayude a los robots a regresar a las estaciones de recarga después de entregar los paquetes. Sin embargo, los robots poseen una cantidad extremadamente pequeña de memoria. Por lo tanto, al diseñar el programa de navegación los robots deberán tomar decisiones basadas únicamente en la cantidad de estaciones de recarga $K = 6$, el grado mínimo B de los nodos no hojas, y los tipos de los cables que salen del nodo donde se encuentra el robot actualmente.

Cuando un robot planea navegar hacia algún nodo u de tipo aldea conectado por al menos dos cables, el robot primero escanea los cables conectados al nodo u en orden arbitrario. El programa recibirá la lista de los tipos de cables sin algún orden específico. Para cada uno de los cables, el

programa deberá calcular el número de estaciones de recarga para las cuales al tomar este cable, la distancia entre el robot y dicha estación decrece.

Formalmente, supongamos que $v_1, \dots, v_k$ ($k \geq 2$) son los nodos directamente conectados a u por un cable y que c_i ($1 \leq i \leq k$) es el tipo de cable que conecta a los nodos u y v_i . El programa recibirá los valores K , B y la lista $c_1, c_2, \dots, c_k$. Para cada i ($1 \leq i \leq k$), el programa deberá determinar el número de estaciones de recarga p tales que $d(v_i, p) < d(u, p)$.

Tu tarea consiste en diseñar una estrategia para asignar los tipos de cables y diseñar el programa de navegación. El puntaje de tu solución dependerá de la cantidad de tipos distintos de cables utilizados (para más detalles revisar el apartado de Subtareas).

Detalles de implementación

Deberás implementar dos funciones, una para asignar los cables y otra para el funcionamiento de los robots.

La función que debes implementar para **asignar los cables** es:

```
std::vector<int> construct_network(int N, int K, int B,
                                  std::vector<int> U, std::vector<int> V,
                                  std::vector<int> P)
```

- N : el número de nodos.
- K : el número de estaciones de recarga.
- B : el mínimo grado garantizado para los nodos **no hojas**.
- U, V : arreglos de longitud $N - 1$ describiendo los cables que conectan la red.
- P : un arreglo de longitud $K = 6$ indicando las estaciones de recarga.
- Esta función se llamará un **máximo 10 000 veces** por cada caso.

Esta función deberá retornar un arreglo T de longitud $N - 1$:

- Indicando que un cable de tipo $T[i]$ conecta a los nodos $U[i]$ y $V[i]$.
- Cada elemento $T[i]$ deberá cumplir que $0 \leq T[i] < 100$.

La función que debes implementar para el **funcionamiento del robot** es:

```
std::vector<int> navigate(int K, int B, std::vector<int> C)
```

- K : el número de estaciones de recarga.
- B : el mínimo grado garantizado para los nodos **no hojas**.

- C : la lista de todos los tipos de cables conectados a algún nodo de aldea, en orden arbitrario.
- Se garantiza que la longitud de C es al menos B .
- Esta función se llamará a lo más 100 000 **veces** por cada caso de prueba.
- Esta función no deberá depender de la estructura original de la red. Es posible que el evaluador reordene las llamadas a esta función para alternarlas entre redes con diferente estructura dentro del mismo caso.

Esta función deberá retornar un arreglo D :

- La longitud del arreglo D debe ser del mismo tamaño que el arreglo C .
- $D[i]$ debe ser igual al número de estaciones de recarga para las cuales al tomar el cable correspondiente a $C[i]$ la distancia entre el robot y la estación decrece.

Toma en cuenta que en el evaluador oficial, las llamadas a `construct_network` y `navigate` se realizan **en dos procesos separados**.

Restricciones

- $K = 6$.
- $K < N \leq 100\,000$.
- La suma de N entre todas las llamadas a `construct_network` no excede 100 000 para cada caso de prueba.
- $0 \leq U[i] < V[i] < N$ para cada $0 \leq i \leq N - 2$.
- Siempre es posible moverse desde cualquier nodo a cualquier otro usando los cables que los conectan.
- $0 \leq P[0] < P[1] < \dots < P[K - 1] < N$.
- La suma de las longitudes del arreglo C entre todas las llamadas a `navigate` no excede 200 000 para cada caso de prueba.

Subtareas

Subtarea	Valor	Restricciones adicionales
1	6	$B = 2, N = 7$.
2	14	$B = 2, U[i] = i, V[i] = i + 1$ para cada i en el rango $0 \leq i < N - 1$.
3	20	$B = 5$.
4	20	$B = 4$.
5	40	$B = 2$.

Para cada caso, si al menos una de las siguientes condiciones se cumple, el puntaje de tu solución para ese caso será 0 (el mensaje dentro del CMS será `Output isn't correct`):

- El valor de retorno para al menos una llamada a `construct_network` es inválido.
- El valor de retorno para al menos una de las llamadas a `navigate` es incorrecto.

En caso contrario, se define S como el entero más pequeño mayor que todos los números en todos los arreglos T devueltos por cada llamada a `construct_network`. Dicho de otra forma, S es el entero más pequeño tal que todas las redes usan cables únicamente de tipos $0, 1, \dots, S - 1$

Tu puntaje en cada subtarea depende de S , y se calcula de la siguiente manera:

Valor de S	Subtarea 1	Subtarea 2	Subtareas 3 y 4	Subtarea 5
$100 < S$	0	0	0	0
$16 \leq S \leq 100$	2	2	$12 - \log_2 S$	$24 - 2 \log_2 S$
$10 \leq S \leq 15$	2	4	$16 - 0.5S$	$32 - S$
$7 \leq S \leq 9$	2	6	$21 - S$	$42 - 2S$
$S = 6$	2	10	15	30
$S = 5$	6	14	17.5	40
$S \leq 4$	6	14	20	40

En particular, para las subtareas 1, 2 y 5 recibirás todos los puntos si $S \leq 5$, mientras que para las subtareas 3 y 4 recibirás todos los puntos solo si $S \leq 4$.

Ejemplo

Considera el caso en el que $N = 9$, $K = 6$, and $B = 2$.

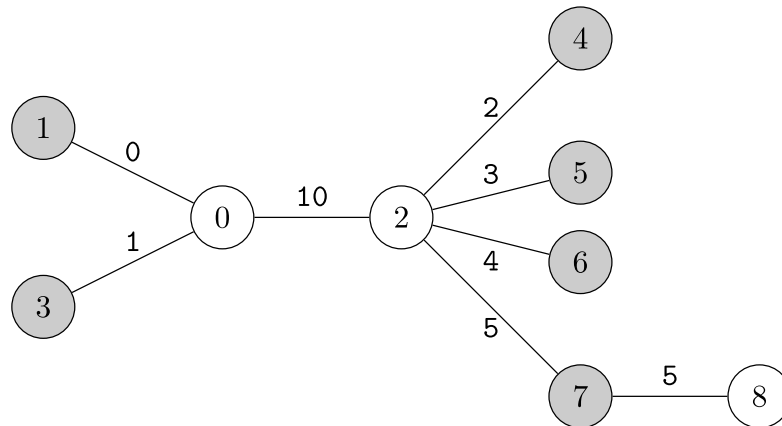
La estructura de la red de entregas está dada por $U = [0, 0, 0, 2, 2, 2, 2, 7]$, $V = [1, 2, 3, 4, 5, 6, 7, 8]$. Las estaciones de recarga son $P = [1, 3, 4, 5, 6, 7]$. Considera la siguiente llamada al primer proceso.

```
construct_network(9, 6, 2, [0, 0, 0, 2, 2, 2, 2, 7],
                  [1, 2, 3, 4, 5, 6, 7, 8],
                  [1, 3, 4, 5, 6, 7])
```

Supongamos que A-Ming asigna los tipos de cable de la siguiente manera:

```
[0, 10, 1, 2, 3, 4, 5, 5]
```

La red resultante se vería de la siguiente manera.



Luego, en el segundo proceso, supongamos que el robot necesita navegar al nodo 0. El nodo 0 está conectado a los nodos 1, 2 y 3. Si el robot lee los tipos de cable en este orden, se realizará la siguiente llamada:

```
navigate(6, 2, [0, 10, 1])
```

Basados en la estructura de esta red:

- Las estaciones de recarga, 1 y 3 son directamente alcanzables por el nodo 0, por lo que se minimiza su distancia al moverse a su respectivo nodo. Al nodo 1 para la estación de recarga en el nodo 1 y al nodo 3 para su estación de recarga en ese nodo.
- Las otras estaciones de recarga, 4, 5, 6, y 7, están más cerca del nodo 2 (directamente alcanzable por el nodo 0) que del nodo 0.

La cantidad de estaciones de recarga para las cuales su distancia decreció al tomar los cables (0,1), (0,2), y (0,3) es 1, 4, y 1 respectivamente. Por lo que se deberá retornar:

```
[1, 4, 1]
```

Ten en cuenta que `navigate` puede ser llamada con diferentes permutaciones de C . Por ejemplo, `navigate(6, 2, [1, 0, 10])` es otra posible llamada para el nodo 0. `[1, 1, 4]` también es el arreglo que se debe retornar para esta llamada.

Supongamos que el robot necesita navegar al nodo 2. Se realiza la siguiente llamada:

```
navigate(6, 2, [10, 2, 3, 4, 5])
```

La función deberá retornar:

```
[2, 1, 1, 1, 1]
```

En esta red, la función `navigate` nunca será llamada para otros nodos que no sean los nodos 0 y 2 debido a que:

- los nodos 1, 3, 4, 5, 6 y 7 son estaciones de recarga, y
- el nodo 8 está conectado a exactamente un cable.

En este caso de prueba, los tipos de cables utilizados son 0, 1, 2, 3, 4, 5, y 10. Por lo tanto, $S = 11$ será el valor utilizado para determinar el puntaje en este caso.

La carpeta de archivos adjuntos para este problema tendrá otros dos casos además de este para el evaluador de local.

Evaluador local

El evaluador local llama a las funciones `construct_network` y `navigate` en una misma ejecución. Además, cuando el evaluador local llama a la función `navigate`, los tipos de cable se dan en el orden en el que aparecen en el caso de entrada. Esto difiere del evaluador oficial.

Formato de entrada:

```
T
(caso 0)
(caso 1)
...
(caso T-1)
```

Cada caso está dado de la siguiente manera:

```
N
B
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
K
P[0] P[1] ... P[K-1]
Q
X[0] X[1] ... X[Q-1]
```

En cada caso, el evaluador local llamará a `navigate` Q veces, la i -ésima llamada será con la información del nodo $X[i - 1]$.

Aunque en los casos oficiales el valor de K está fijo, el evaluador local sí admite casos en los que el valor de K es distinto de 6.

Formato de salida:

Si alguno de los arreglos por las llamadas `construct_network` o `navigate` son inválidos, el evaluador de ejemplo terminará la ejecución e imprimirá un mensaje con el respectivo error. En caso contrario, el evaluador de local imprimirá el arreglo D retornado por la función `navigate` en una sola línea.

Después de haber procesado todos los casos, el evaluador de ejemplo imprimirá un valor:

```
S = S'
```

donde S' es el mínimo entero mayor a todos los números de cada arreglo T retornado por la función `construct_network` en el caso de prueba.